Améliorations du pipeline d'extraction des textes juridiques du Bulletin Officiel

Récapitulatif des bugs corrigés, de l'intégration de Gemma pour la vérification sémantique, et de l'adaptateur pour les fichiers JSONL (OCR par blocs).

Vue d'ensemble

Le pipeline original (start_pattern.py / end_pattern.py / arabic_legal_extractor.py) détecte les débuts de textes juridiques par regex (mot-clé + numéro + date), détecte les fins via les blocs de signature, puis associe chaque début à la fin la plus proche. Ce document résume les corrections de bugs et les ajouts effectués, avec des extraits de code avant/après et les résultats mesurés sur les vrais fichiers du Bulletin Officiel.

Fichier	Composant	Type de changement
start_pattern.py	Détection des débuts	3 bugs corrigés + robustesse
arabic_legal_extractor.py	Orchestrateur	1 bug de position corrigé + format de sortie
gemma_client.py	Nouveau	Client HTTP vers le modèle Gemma auto-hébergé
gemma_verifier.py	Nouveau	Vérification sémantique des candidats (parallèle)
jsonl_adapter.py	Nouveau	Adaptateur JSONL (OCR par blocs) + traitement par lot

1. Bugs corrigés — start_pattern.py

1.1 Bricolage de positions remplacé

L'ancien code simulait un objet re.Match avec type('Match', (), {...})() pour recalculer des positions absolues. Remplacé par des entiers simples, plus lisible et moins fragile. Les positions (start_position / end_position) sont maintenant stockées directement dans chaque résultat, ce qui permettra de corriger le bug de la section 2.

```
# Avant
match_with_position = type('Match', (), {
    'start': lambda self: line_start + match.start(),
    'group': lambda self, key: match.group(key)
})()
result = extract_sequential_info(text, match_with_position)

# Apres
kw_start = line_start + leading_offset + match.start()
kw_end = line_start + leading_offset + match.end()
result = extract_sequential_info(text, match.group('keyword'), kw_start, kw_end)
```

1.2 Crash sur les candidats sans vrai numéro

Un mot-clé sans numéro à proximité (ex: قرار بلدي يقضي بالتصفيف) faisait partir la recherche de عدد/رقم très loin dans le texte, jusqu'à tomber parfois sur l'en-tête d'un tableau markdown sans rapport. Le résultat renvoyé était un dictionnaire incomplet (il manquait les clés 'keyword', 'full_match'...), ce qui provoquait un KeyError plus loin dans le pipeline sur le vrai fichier BO_3409.

```
# Nouvelle constante : distance maximale pour chercher un numero
NUMBER_INDICATOR_SEARCH_WINDOW = 80

search_window = remaining_text[:NUMBER_INDICATOR_SEARCH_WINDOW]
number_indicator_match = re.search(pattern, search_window)

if len(re.findall(r'\d', number_text)) < 1:
    # Avant : return {'number_text': number_text.strip()}  -> dict incomplet, crash
    return None  # Apres : candidat proprement ecarte
```

Impact mesuré : le script plantait avec KeyError: 'keyword' sur BO_3409.jsonl avant correction ; il traite maintenant les 6 fichiers du lot sans interruption.

1.3 Indicateur de date manquant (impact le plus important)

DATE_INDICATORS ne contenait que des variantes de صادر في. Or de nombreux textes réels du BO utilisent بتاريخ pour introduire la date (ex : 1398 ربيع الأول 19 بتاريخ 2.78.60 مرسوم رقم). Ces textes réels étaient silencieusement écartés (aucune erreur, juste absents du résultat).

```
DATE_INDICATORS = [
    r'\u0635\u0627\u062f\u0631\s+\u0641\u064a',   # sadir fi
    ...
    r'\u0628\u062a\u0627\u0631\u064a\u062e\s*',   # NOUVEAU : bitarikh
    '\u0627\u0644\u0645\u0624\u0631\u062e \u0641\u064a',   # NOUVEAU : al-mou'arrakh fi
]
```

Impact mesuré sur BO_3409.jsonl : 0 → 76 candidats de début « avec signature » détectés, 0 → 60 documents juridiques complets extraits.

2. Bug corrigé — position dupliquée (arabic_legal_extractor.py)

L'ancien code retrouvait la position d'un début déjà détecté avec content.find(full_match). Or .find() renvoie la PREMIÈRE occurrence dans tout le document. Sur BO_3334.jsonl, le sommaire (فهرست) du numéro répète le début d'un texte réel presque mot pour mot : les deux occurrences pouvaient alors se voir attribuer la même position (celle de la première, dans le sommaire), corrompant silencieusement le découpage du vrai texte.

```
# Avant (arabic_legal_extractor.py)
pos = content.find(full_match)   # renvoie TOUJOURS la 1ere occurrence

# Apres : la position est calculee UNE FOIS pendant le scan (start_pattern.py)
# et reutilisee telle quelle, sans jamais refaire de .find()
start_positions = [
    {'position': p['start_position'], 'end_position': p['end_position'], ...}
    for p in start_patterns
]
```

Vérifié sur un cas de test reproduisant exactement ce scénario : l'ancienne logique aurait fait pointer le vrai texte (position réelle 102) vers la position du faux départ du sommaire (position 21) ; la nouvelle logique donne la bonne position dans les deux cas.

3. Ajout de la vérification sémantique par Gemma

Le regex reste la première passe (rapide) pour proposer des candidats de début et de fin. Gemma (modèle interne, servi via vLLM) sert de filtre de confirmation : pour chaque candidat, on lui envoie le texte + son contexte (250 caractères avant/après), et on lui demande de répondre en JSON si c'est un vrai début/une vraie fin, ou un faux positif (ex : une entrée de sommaire).

3.1 gemma_client.py — appel au modèle

```
def call_gemma(messages, temperature=0.0, max_tokens=300):
    response = requests.post(
        GEMMA_API_URL,
        headers={'Authorization': f'Bearer {api_key}'},
        json={'model': GEMMA_MODEL, 'messages': messages,
              'temperature': temperature, 'max_tokens': max_tokens},
        verify=GEMMA_VERIFY_SSL,
    )
    return response.json()['choices'][0]['message']['content']
```

3.2 gemma_verifier.py — prompt de vérification

Le prompt (en arabe, adressé au modèle) décrit explicitement le cas piège trouvé sur BO_3334 : une entrée de sommaire répète un mot-clé + numéro + date, mais n'est pas un vrai début.

```
def verify_start_candidate(full_text, candidate):
    before, after = get_context(full_text, candidate)
    prompt = START_VERIFICATION_PROMPT.format(
        before=before, candidate=candidate['full_match'], after=after)
    try:
        raw = call_gemma([{'role': 'user', 'content': prompt}])
        result = extract_json_object(raw)
        candidate['gemma_verified'] = bool(result.get('is_valid_start', True))
    except (GemmaClientError, ValueError) as e:
        # fail-open : on garde le candidat plutot que de le perdre
        # silencieusement si Gemma est indisponible
        candidate['gemma_verified'] = None
    return candidate
```

Politique fail-open : si l'appel échoue (réseau, timeout, JSON invalide), le candidat est gardé (gemma_verified = None) plutôt que supprimé silencieusement. Seuls les candidats explicitement rejetés (gemma_verified = False) sont écartés.

3.3 Parallélisation des appels

Testé en conditions réelles : sur BO_3405.jsonl (249 candidats de début + 44 de fin), vérifier en série donnait l'impression que le script était bloqué. Passage à un pool de threads (les appels sont uniquement en attente réseau) avec affichage de progression.

```
MAX_PARALLEL_CALLS = 5

with ThreadPoolExecutor(max_workers=MAX_PARALLEL_CALLS) as executor:
    future_to_index = {
        executor.submit(verify_start_candidate, full_text, c): i
        for i, c in enumerate(candidates)
    }
    for future in as_completed(future_to_index):
        verified[future_to_index[future]] = future.result()
        done_count += 1
        if done_count % 10 == 0:
            print(f'... {done_count}/{total} verifies')
```

4. Nouveau — jsonl_adapter.py

Le format réel fourni n'est pas un fichier texte avec des marqueurs '=== Page N ===', mais un JSONL : une ligne par page, avec une liste de blocs OCR (coordonnées x/y et un type : header, title, text, table, list...). Cet adaptateur reconstruit le texte continu attendu par le pipeline, sans changer sa logique de détection/matching.

```
def build_content_from_pages(pages):
    for page in pages:
        marker = f'\n=== Page {page["page"]} ===\n'
        content_parts.append(marker)
        page_positions.append({'page_number': page['page'], 'position': cursor})
```

```
        cursor += len(marker)
        for block in page.get('blocks', []):
            content_parts.append(block['content'] + '\n\n')
            cursor += len(block['content']) + 2
    return ''.join(content_parts), page_positions, block_positions
```

arabic_legal_extractor.py a été restructuré pour séparer la lecture du fichier (extract_legal_documents) du cœur du pipeline (extract_legal_documents_from_content), que jsonl_adapter.py appelle directement — la logique de détection/matching n'est écrite qu'une seule fois pour les deux formats d'entrée.

4.1 Traitement par lot (dossier /input)

process_folder() boucle sur tous les .jsonl d'un dossier. Un fichier corrompu ou en échec est logué et n'interrompt pas le traitement des autres.

```
for jsonl_file in sorted(Path(input_dir).glob('*.jsonl')):
    try:
        documents = extract_legal_documents_from_jsonl(str(jsonl_file), use_gemma)
        save_extracted_documents(documents, output_path)
        summary[jsonl_file.name] = len(documents)
    except Exception as e:
        print(f'Echec sur {jsonl_file.name} : {e}')
        summary[jsonl_file.name] = -1   # on continue avec le fichier suivant
```

5. Format de sortie personnalisé

save_extracted_documents() a été réécrit pour reproduire le format demandé (bannière, un bloc « TEXT N » par document avec métadonnées à puces, séparateurs), avec deux ajouts par rapport à l'exemple fourni :

- un champ « Page » explicite pour CHAQUE document (y compris mono-page, pour lesquels aucun marqueur de page n'apparaît naturellement à l'intérieur du texte) ;
- un décalage +1 à l'affichage : la page 0 du JSONL correspond à la page 1 du PDF réel. Le décalage n'est appliqué qu'à l'affichage, jamais dans les calculs internes de position.

```
def format_page_label(doc):
    start_display = doc['page_number'] + 1       # decalage +1 ici seulement
    end_display = doc.get('end_page_number')
    if end_display is None or end_display == doc['page_number']:
        return str(start_display)
    return f'{start_display}-{end_display + 1}'
```

Exemple de sortie (document réel, BO_3405)

```
TEXT 5

- Keyword: qarar
- Number: 815.77
- Date: 5 joumada ath-thania 1398 (4 juin 1976)
- Key Signature: al-imda'
- Names: Abdelkader Bensliman
- Page: 9-12          <-- verifie : correspond aux vraies pages 9 a 12 du PDF
```

6. Résumé chiffré (mesuré sur les vrais fichiers)

Fichier	Avant corrections	Après corrections

BO_3409.jsonl	0 document avec signature (بتاريخ non reconnu)	76 candidats confirmés, 60 documents extraits
BO_3405.jsonl	Crash KeyError (candidat sans numéro)	195 documents extraits (93 avec signature, 156 sans)
BO_3334.jsonl	Sommaire capturé comme faux départ + position erronée	Position correcte pour chaque occurrence

7. Limites connues / prochaine étape suggérée

Les textes SANS signature (annonces, إعلان) peuvent encore fusionner plusieurs entrées non liées en un seul document : leur fin est déterminée par « le prochain mot-clé AVEC signature trouvé », qui peut être très loin si plusieurs annonces se suivent. Cette limite touche uniquement les textes sans signature — les textes avec signature (قرار, مرسوم, ظهير) sont correctement délimités, y compris sur plusieurs pages. Correctif proposé (non encore appliqué) : arrêter aussi sur le prochain mot-clé SANS signature, et faire passer ces limites de fin par la vérification Gemma comme les autres.